Objects First with Java: Correctness Supplement

Anne Mulhern

Introduction

Programmers should be taught how to write correct programs. However, correctness is hard, and teaching correctness is harder still. Consequently, instruction in writing programs correctly is too often omitted from introductory programming courses.

Students seldom learn how to think about correctness in subsequent courses. Software engineering courses are too often devoted to the nebulous practice of "project management" rather than the discipline of ensuring correctness. Courses explicitly devoted to ensuring the correctness of programs focus on formal methods. Unfortunately, what is taught in these courses is seldom applied elsewhere in a curriculum. Moreover, the tools used in these courses are often research projects which frequently:

1. Are difficult to install and use.
2. Work only with an obsolete version of the object language.
3. Are poorly documented.
4. Report results that are inscrutable to the novice programmer.
5. Require a tremendous investment in the study of logic before they can be of any use.

Under these circumstances students are likely to learn that program correctness is a chimera and to associate the topic of program correctness with tools that are nearly impossible to understand and use.

BlueJ[BlueJ] is a Java IDE designed especially for teaching. It is quite mature and has an intuitive interface. Particularly valuable are its Object Bench, Code Pad, debugger, interface with JUnit[JUnit], and extension facility. Greenfoot[Greenfoot] is an object world for Java which allows a user to develop simple animations rapidly.

This work is a supplement to "Objects First with Java" by David Barnes and Michael Kölling otherwise known as "the BlueJ book"[BK2012]. It makes use of many BlueJ features and uses Greenfoot as a wrapper for numerous projects. It gradually introduces techniques for ensuring program correctness in parallel with the chapters of the textbook. The techniques are chosen for their stability, simplicity, clarity, and effectiveness and it is expected that students will be able to make use of them in subsequent courses.

Part I

Foundations of object orientation

Chapter 1

Objects and classes

1.1 Creating Unit Tests Interactively

In this chapter we have learned how to invoke methods interactively. We can use the same technique to interactively build tests that can be saved and re-run whenever we change our code.

1.1.1 First tests

Open the *lab-classes* project. From the `Student` class's context menu select `Create Test Class.`

A new class, called `StudentTest` will appear. Note that its color differs from that of other classes and that it contains the special annotation `<<unit test>>`. A unit test is a test that verifies the behavior of a method in isolation from the behavior of other methods. We will construct unit tests for every method in the `Student` class. To see what those methods are, construct a `Student` object and examine the methods that are available for that object. We will begin with methods that start with "get", since they do not depend on any other methods.

From the `StudentTest` context menu select `Create Test Method.`

We will begin by testing `getCredits()`, so specify `testGetCredits` as the name of the test method. Make a new `Student` instance called `asterix` with name "Asterix" and id "320". Select the instance and choose `getCredits()` from the context menu. Since Asterix has yet to take any classes he should have 0 credits, so put 0 in the empty field and make sure that `equal to` is selected from the adjacent menu. Observe that the recording light is red. This is about all the testing such a simple method should require, so press `End` to stop recording. Notice that the `Run Tests` button is enabled. Press it and observe that your one test runs and passes.

Exercise: Write similar tests for each of the other "get" methods in the `Student` class.

1.1.2 Testing command methods

`changeName()` is a method that changes the state. We will test the name by invoking it on a `Student` instance and then checking that the name of the instance has been changed appropriately. Begin by creating a test method and naming it `testChangeName`. Create a new `Student` called `asterix` with name "Asterix" and id "320". Invoke the `changeName()` method on the object using the actual parameter "Asterix the Gaul". Test that the name is now changed by invoking the `getName()` method on the object. Enter "Asterix the Gaul" in the input field and select `equal to` from the menu.

Verifying that the name has changed to the correct value is not quite good enough. It is also important to make sure that other parts of the instance's state have not changed. To verify this we should use the other "get" methods. Invoke the `getStudentId()` method on the object and enter "320" in the appropriate field. Make sure to verify that the result of `getCredits()` also remains the same, i.e., is 0.

Press the `End` button once the test is complete.

Exercise: `addCredits()` is another method that changes state. Write a complete test that verifies not only that the number of credits is changed appropriately after `addCredits()` is called, but also that nothing else has changed. Press the `Run Tests` button to verify that all your tests pass.

1.1.3 Observing your code

If you examine the source code of your `StudentTest` class you will see that each test you made is really just a method of the class. Observe that the methods are annotated, i.e., just above each method is the annotation `@Test`. The JUnit testing framework uses this annotation to identify test methods.

1.2 Comments

Observe that the `Student.java` file in the example that accompanies this supplement contains no comments. Too often, the writing of comments distracts from the pursuit of correctness. Comments are, by definition, the parts of the program that are ignored by the compiler. Quite sensibly, programmers follow the compiler's lead in ignoring comments, which are therefore often stale or incorrect as soon as they are written. Novice programmers tend to be prolix and uninformative in their comments. For example, the comment `Iterate` just before a loop, the like of which I have seen many times in student code, is both redundant and entirely uninformative.

The tests that you have just written are superior to comments as documentation because they are *machine-checkable*. If a test fails, it is the programmer's responsibility to review the test and decide whether it is the test itself which is broken or the object code. In either case, the programmer will take action,

either to correct the test or the object code. Comments do not have such a self-regulating mechanisms.

At some time during your career you are likely to find yourself in a situation where you are required to write comments. In that case, you should adhere to the comment writing guidelines in place where you work. The time you spend in college is too precious to waste on learning this tedious activity. On the other hand, if, by some chance, you already have the habit of writing useful comments do not desist now. Your adjustment to a workplace where commenting is enforced will be that much easier.

1.3 Formatting Your Code Correctly

Well formatted code is easier to read and consequently to understand. Understanding is a prerequisite to correctness; therefore good formatting aids correctness. Checkstyle[Checkstyle] is a tool that identifies instances where a programmer has failed to adhere to a coding standard. Checkstyle is configurable to different coding standards.

1.3.1 Running Checkstyle in BlueJ

Checkstyle is available as a BlueJ extension and must be added to BlueJ before it can be used. Once it is available in BlueJ a `Checkstyle` choice will be available in the `Tools` menu.

Open the *lab-classes* project. Select `Checkstyle` and you will see a new window that identifies all the formatting errors that Checkstyle has located in the project. You will see that the Checkstyle extension uses a particular default configuration file that places a good deal of emphasis on comments. Instead of using the default configuration, change your configuration so that it refers to the file `default_checks.xml` included with this project. Run Checkstyle again and you will see that it no longer reports comment-related errors.

Continue to make use of this configuration file for every other project that you submit. You will lose points if you submit programs with Checkstyle errors.

Exercise: Open some book projects and run Checkstyle on these. You will probably encounter quite a few errors.

1.3.2 final

Your Checkstyle configuration requires the use of the keyword `final` for every local variable that is unmodified. In Java, all formal parameters are local variables. Notice that no formal parameter in any method in the `Student` class is modified so all are declared `final`.

By using the `final` modifier you communicate to all future readers of your code, and to the Java compiler, that the modified variable is a name for a value and is not to be assigned to.

Note that the instance variable `id` is also declared final. It is set once in the constructor and then may not be changed subsequently.

In general your code will be easier to understand if it contains fewer assignments; all other things being equal, the frequent occurrence of the `final` keyword is a good sign.

1.3.3 this

Every method in the `Student` class takes an implicit parameter, `this`. `this` is a reference to the particular object on which the method has been invoked. All methods that work in this way are called instance methods. Other sorts of method will be introduced when the need arises.

Whenever any method uses an instance variable of the class we specify that instance variable by the `this` keyword, followed by a period, followed by the name of the instance field. For instance, in the `getStudentID()` method the instance field `id` is specified by `this.id`. Your Checkstyle configuration requires that all instance fields be specified using `this`. Within the tiny methods of the `Student` class the use of `this` is almost redundant since there are no locally declared variables other than final parameters; in larger methods, however, the use of `this` serves to distinguish the instance variables from locally declared variables where they are used.

1.4 Summary

In this section you have learned how to construct simple unit tests for the methods of a class. Unit tests have a value that comments lack. Format your code consistently so that you, and others, can understand it more easily. Use `final` in every case where your variables are intended to be immutable. Use `this` whenever you are referring to an instance variable in order to visually distinguish instance variables from locally declared variables.

Chapter 2

Understanding class definitions

2.1 Creating Unit Tests by Modifying the Source

When we build tests interactively using BlueJ testing code is generated in a special testing class. After we become more familiar with testing we will find that simply writing the test code directly is quicker and more convenient. In this exercise we will write some simple tests for the `Book` class.

Open the *book-exercise* project and create a test class for the `Book` class. Begin by interactively generating a test case for the Book class for the `getAuthor()` method. Create a new instance of `Book` called `ww`, and initialize it with the actual parameters "Diana Wynne Jones", "Witch Week", and 273. Then invoke the `getAuthor()` method on the instance and check that the result is equal to "Diana Wynne Jones".

The `getAuthor()` method has not been implemented correctly; your test should fail. Once you have finished verifying that the test fails, open the source code file for the `BookTest` class.

You should see a method called `testGetAuthor()` that looks like

```
@Test
public void testGetAuthor()
{
    Book ww =
        new Book("Diana Wynne Jones",
                 "Witch Week",
                 273);
    assertEquals("Diana Wynne Jones", ww.getAuthor());
}
```

Observe that the code performs exactly the steps that you performed interactively. In the first line of the method a new `Book` is instantiated. In the second

line the `assertEquals()` method is used to check whether "Diana Wynne Jones" is equal to the result of `ww.getAuthor()`.

Notice also that an annotation, `@Test`, appears just above the method. This annotation is used by the JUnit testing framework to identify test methods.

Finally, all instance variables are `final`.

Exercise: Write another method to test the `getTitle()` method. This time, rather than using the recording facility, enter the test as text in the editor window. This test, too should fail. Once you finish both tests and verify that they both fail, edit the `Book` class so that they succeed.

Finally, write a test for the `getNumPages()` method. Verify that the test fails, fix the `getNumPages()` method so that it is correct, and then verify that all your tests succeed.

2.2 Ticket Machine

2.2.1 Overview

In this section we present an alternative approach to the design of the ticket machine in the BlueJ textbook. We alter the design slightly and show how we can make use of testing in our design. We have built a simple graphical interface to the ticket machine in Greenfoot. If the `TicketMachine` class is implemented correctly, all that is necessary is to place the implementation in the Greenfoot *ticket-machine* project and the entire implementation will work appropriately.

2.2.2 Exceptions

Observe that the `TicketMachine` constructor and the `insertCoin()` method make use of exceptions. Exceptions are used to signal errors that may occur at run time but can not be detected at compile time.

Exercise: Open the *ticket-machine* project and construct a `TicketMachine` object passing a `boolean` argument, e.g., `true`, for the price. Observe that BlueJ will not attempt to execute the method invocation because the Java compiler has detected that the type of the actual parameter, `boolean`, is incompatible with the expected type of the parameter, `int`. Try again, this time using a negative value, e.g., `-2`, for the ticket price. Observe that an `IllegalArgumentException` is thrown.

2.2.3 Types and values

The type of a formal parameter designate the set of values which may be passed as an argument in the place of that parameter. You can discover the type of any value, in fact of any expression, by entering it in the Code Pad. A negative ticket price, which would denote a situation in which the person who purchases

a ticket receives some money in addition, is forbidden by our design. However, `int` includes both negative and positive integers. Consequently, the compiler is unable to detect an error and the code makes use of an exception to catch the error at runtime instead.

Exercise: Examine the following values and expression and predict their type. Note that it is always possible to correctly predict the type of an expression without executing the expression, i.e. at compile time. Enter each expression in the Code Pad and verify your prediction.

- `2`
- `-2`
- `2.5`
- `true`
- `2.5 + 2`
- `0 / 2`
- `new TicketMachine(2, 0)`
- `new TicketMachine(3, 0).getBalance()`
- `new TicketMachine(-3, 2)`

The last is tricky; the type of the expression is `TicketMachine`, but since the constructor throws an exception instead of terminating BlueJ can not evaluate the expression and Code Pad does not report a type. Drag your `TicketMachine` object icon from the Code Pad onto the Object Bench. Observe that the Object Inspector icon also indicates the type of the object, i.e., `TicketMachine`.

2.2.4 Using Assertions

Exceptions guard against misuse of an object by others. Since the `TicketMachine` constructor throws an exception whenever the value of the `ticketPrice` argument is less than 0 all classes that use a `TicketMachine` object must obey the `TicketMachine`'s contract by making sure to call its constructor with non-negative arguments.

The arguments of the constructor are used to specify its `ticketPrice` and `total` instance variables. Neither of these fields should ever become less than 0. We can prevent external code from setting these values incorrectly by means of exceptions. However, we also wish to prevent internal code, i.e., methods in `TicketMachine` from setting the values of these instance fields incorrectly.

Any fact that must hold for an object throughout its existence is called a class invariant. Our design requires that the values of `ticketPrice`, `total`, and `balance` always be non-negative. Thus, our class invariant is

$$this.ticketPrice \geq 0 \wedge this.total \geq 0 \wedge this.balance \geq 0$$

We use assertions to detect incorrect values of instance fields. Every time a method is called the class invariant is checked by means of an assertion. If the expression evaluates to false, an `AssertionError`, a special type of exception, is thrown.

The purpose of assertions is not to detect the errors of other in using our code. Their purpose is to keep the programmer honest and to give early warning of an incorrect assumption in the design. During development, assertions should be enabled so that errors are detected. When code is deployed assertions are generally disabled. However, if a bug is reported an investigating programmer can re-enable assertions and use them to detect if some underlying assumption in the code has been violated.

Note that assertions, like unit tests and unlike comments, are *machine-checkable* documentation.

Exercise: Try out some assertions in the Code Pad. Some suggestions are:

- `assert(true);`
- `assert(false);`
- `assert(2 > 1);`
- `assert((1.0 / 3.0) * 3.0 == 1.0);`
- `assert((1 / 3) * 3 == 1);`

Note that an assertion is a statement and so must be terminated with a semicolon. Moreover, the assertion expression must always have type `boolean`.

2.2.5 Writing Interactive Tests

Exercise: Write tests for the "get" methods in the `TicketMachine` class. Run them, to make sure that they fail and then implement the "get" methods correctly.

Exercise: Write a test that verifies that the balance changes by the appropriate amount and no other instance variables change at all when `insertCoin()` is called. Once you have run the test, implement the `insertCoin()` method correctly.

2.2.5.1 Combination methods

The `dispenseRefund()` method is both a command method and a question method. It must perform the following actions:

- Return the current value of the balance.
- Set the current value of the balance to 0.

The `dispenseChange()` method is both a command method and a question method as well. It must perform the following actions:

- Return the appropriate change amount. The change amount may be 0, positive, or negative. A negative value indicates that the balance is less than the ticket price.
- If a ticket can be dispensed, i.e., if the balance is at least the amount of the ticket price:
  - Reset the balance to zero.
  - Increase the total by the price of a ticket.

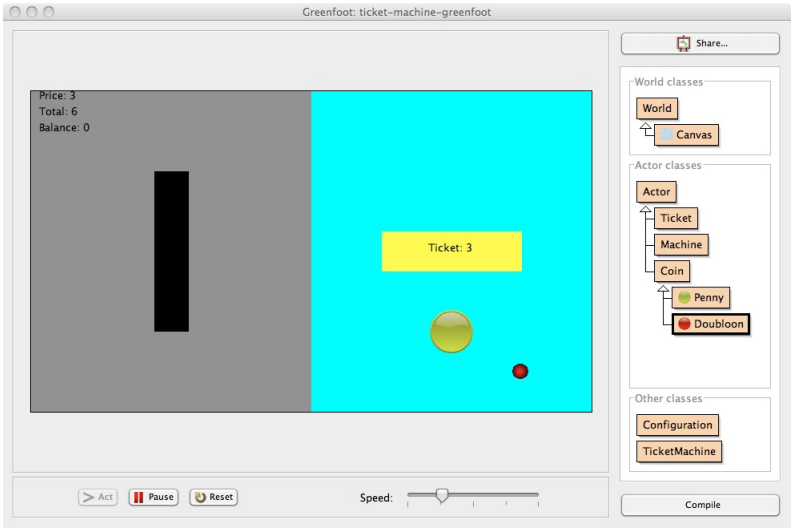
In general, it is poor design to write a method that is both a command method and a question method. However, in this case there is a compelling reason to design the method this way; the calculation of the refund depends on the same conditions as the updates to the different instance variables.

Exercise: Write a test for `dispenseRefund()` that verifies that the balance changes to 0, that the amount returned is correct, and that no other instance variables change their value. Once you have run the test, implement the `dispenseRefund()` method correctly.

Exercise: Write two tests for the `dispenseChange()` method. Make sure to test the behavior of `dispenseChange()` when the balance is less than the price and also when the balance is greater. Remember that you must test the values of all the instance variables, those that should change and those that should remain the same. Once you have written all your tests, implement the `dispenseChange()` method correctly.

2.2.6 Transferring to Greenfoot

Now that you have completed the `TicketMachine` class open the *ticket-machine-greenfoot* project in Greenfoot. The version of `TicketMachine` in this project is the incomplete version that you started with. Delete this version and copy the version you just wrote. If you have implemented `TicketMachine` correctly the Greenfoot project should work correctly. You should expect it to look like this.



2.3 Summary

Exceptions are used to detect errors that may occur at runtime that can not be detected by the compiler. In this example, it was possible for external code to call a method with a value of the correct type but the wrong sign. An `IllegalArgumentException` should be thrown if a method is invoked with an unsuitable argument. Assertions are used to detect errors in the internal implementation of the code and can be disabled in production code. Exceptions and assertions both serve as code documentation. The interactive test generation mechanism in BlueJ is easily skipped if you prefer to enter your tests as text. When testing command methods it is important to check the values of all instance variables, not just the ones that should change, but also those that should not.

Chapter 3

Object interaction

3.1 Clock

3.1.1 Overview

In this section we present an alternative approach to the design of the clock in the textbook. The key change in our design is that we separate the data from the display of the data. We distinguish the clock's computational part, i.e., the recording and measuring of time from its display part. In the example in the BlueJ textbook, the display of the clock determined how time was computed. Since the clock displayed hours and minutes, two `NumberDisplay` objects were used to store the clock's hours and minutes respectively. We can change our perspective by realizing that hours are just a convenient grouping for the minutes, just like dozens are a convenient grouping for eggs. This means that we can make a clock that simply records the number of minutes that have elapsed.

3.1.2 The clock package

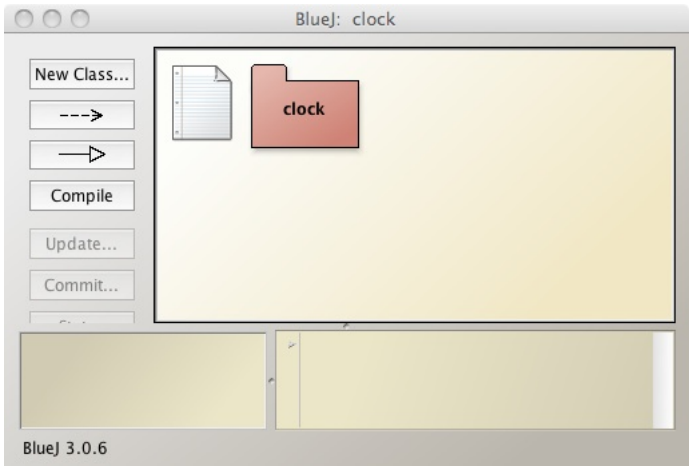
For the computational part we make use of two classes, `Clock` and `Counter`. These classes are grouped in the `clock` package. A package is a higher level of organization that the Java language provides. We choose to group the two classes together because the `Clock` class relies entirely on the `Counter` class and because we do not want any users of our `clock` package to make use of the `Counter` class independently, i.e., not by making a `Clock` object.

Observe the `package` declaration at the top of both files. This declaration is necessary so that the compiler will recognize that the classes belong to the `clock` package. Moreover, note that the files are contained in a directory called `clock`. Java requires that the directory in which the files are contained has the same name as the package to which the files belong.

Observe also that the `Counter` class declaration is not public. This means that only classes that belong to the package can make use of it. The `Clock` class is public, so all external users of the package can use this class.

Exercise: Open the *clock* project.

Make sure that you are outside the `clock` package. Your window should look like the one below with only the `clock` package icon visible.



Create a `Clock` object in the Code Pad. Note that since the code you write in the Code Pad is external to the package you must use the fully qualified name for `Clock`'s constructor, i.e., `clock.Clock`. The fully qualified name is constructed from name of the package to which the class belongs, followed by a period, followed by the class name. Enter `new clock.Clock(3, 5)` in the Code Pad and drag your result to the Object Bench. You can call methods on the object in the normal way. The `Counter` constructor also takes two `int` arguments. Try constructing a `Counter` object with the same arguments as the `Clock` object. Observe that you get an error; since the `Counter` class is package private you can not construct a `Counter` object from outside the package.

Now click on the `clock` package icon to go inside the `clock` package. Any code that you type in the Code Pad is executed within the `clock` package so you can create `Counter` and `Clock` objects with or without using their fully qualified names. Try all possibilities, e.g.

- `new clock.Clock(3, 2)`
- `new Clock(3, 2)`
- `new Counter(3, 2)`
- `new clock.Counter(3, 2)`

3.1.3 The Clock class

The `Clock` object uses a single `Counter` object which counts the minutes that have elapsed since the start of the clock's day. After the total number of minutes in a single day have elapsed, the counter automatically starts over again from 0.

The Clock object interprets the counter's minutes in terms of hours and minutes in the day.

The implementation of `Clock` is complete and a reasonably complete test class has also been constructed. Some of the tests pass; these are test methods of the `Clock` class that do not depend on methods of the `Counter` class. Some of the test fail; these are test methods of the `Clock` class that depend on methods of the `Counter` class.

Notice that the `Clock` constructor takes two parameters: the number of hours in a cycle and the number of minutes in an hour. Methods of this class make use of the instance variables `hoursPerCycle` and `minutesPerHour` wherever appropriate. Observe how the constructor makes use of them in calculating the number of minutes in a cycle to pass as an argument to the `Counter` constructor. By parameterizing the clock in this way it can be made much more general. Choose the hours per cycle to be 12 and the minutes per hour to be 60 and it is a civilian time clock. Change the hours per cycle to be 24 and this clock calculates military time. Change the hours to 180 and perhaps this clock computes the time on an alien planet that takes much longer to complete a single rotation than does our earth.

Observe that all instance variables of the `Clock` class have been declared final; only some of the `Counter`'s instance variables can change.

3.1.4 The Counter

The implementation of `Counter` is incomplete; your task is to complete it. We observe that the Counter class has an invariant:

$$this.limit > 0 \wedge 0 \leq this.value < this.limit$$

Remember that the `Counter` class is package private; only code within the package can call the constructor. Therefore, we should use an assertion, rather than exceptions, to detect bad argument values. The assertion condition is just like the class invariant except that it is a statement about the values of the parameters, not the instance variables. If the parameters are invalid then we should not construct the object, so a condition about the instance variables is pointless.

$$limit > 0 \wedge 0 \leq value < limit$$

Exercise: Add an assertion at the start of every method of the `Counter` class to ensure `Counter`'s class invariant.

Add an assertion at the start of the `Counter` constructor to ensure that every `Counter` object is constructed correctly.

Develop a set of tests for each method of the `Counter` class. The two "get" methods are obvious; the tests should be very simple. Make sure your test for the `increment()` method verifies that the counter rolls over properly when the limit is reached. Initially, your tests should fail. Once you have developed your

tests, implement each method correctly. All your tests should succeed, including the tests for the `Clock` class.

3.1.5 Testing the clock package

After constructing package local unit tests you should also construct unit tests that are external to the package.

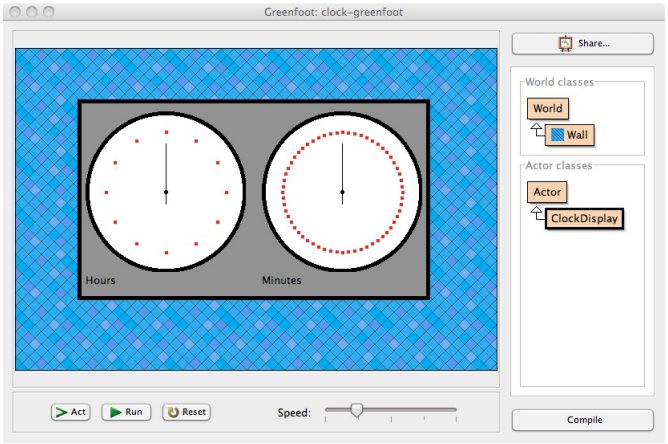
Exercise: Navigate within BlueJ so that the project is external to the package. Construct a new test class called `ClockTest`. Note that this class is distinct from the other `ClockTest` class that you have already constructed because it is not a member of the `clock` package. Write a set of appropriate tests. You may need to make use of CodePad to build your objects; the unit test recording facility will record your Code Pad actions.

3.1.6 Transferring to Greenfoot

When you are done, the computational part of your clock is complete. However, it has no display at all. To incorporate your clock implementation into a display open the Greenfoot project *clock-greenfoot:*

1. Use BlueJ to create a jar file of your package.
2. In Greenfoot's Preferences window choose the Libraries tab and select the jar file you made in the previous step.

You should expect your project to look something like this:



3.2 Summary

Java packages are used to assemble Java classes into groups of cooperating classes. A class that is package private can not be accessed by code outside the

package. It is important to test both within a package and outside a package. The class invariants that we devise have become more complex, specifying both an upper and a lower bound. You can store an entire package as a jar file to make the public contents of the package available to other projects.

Interlude 1: Formal specifications (incomplete)

Overview

The previous chapters have described approaches to unit testing. Unit testing is very useful because:

1. Unit tests are a specification of a program's behavior.
2. Unit tests, once written, automatically locate errors that may be introduced when a program is implemented or modified.

As specifications of behavior, however, unit tests are primitive, because, rather than specifying the behavior of a method in every situation, each test specifies the behavior of a method in a very small set of situations. Formal specifications are an application of mathematical logic that can be used to succinctly specify the behavior of a method in all circumstances.

This chapter describes the use of pre- and post-conditions to specify the behavior of methods. It discusses the ways in which unit tests can be written so that they perform their first role, functioning as a specification, more effectively.

The chapter assumes that the reader is familiar with implication as it is understood in classical propositional logic.

Predicates as sets

Consider the `Counter` class from the previous `clock` project. How many `Counter` object with a limit of 3 are possible? There are only 3, since the only possible values for a `Counter` object with a limit of 3 are 0, 1, and 2.

Exercise: Open your completed `clock` project and construct all possible `Counter` objects with a limit of 3.

Consider the the following logical formula

$$\text{this.limit} = 4$$

Does this formula hold for any of the `Counter` objects that you have created? The answer is no, since all have 3 as the limit, so, of the `Counter` objects that you have created, the subset described by your logical formula is the empty set.

The following logical formula

$$\text{this.limit} = 3 \wedge \ \text{this.value} = 4$$

is impossible, since the `Counter` constructor ensures that no `Counter` object with a value greater than its limit can be constructed.

Other formulas select a subset of `Counter` objects. For example, the formula

$$\text{this.value} + 1 = \ \text{this.limit}$$

describes all `Counter` objects that will roll over if the `increment()` method is called. In the next section we use logical formulas like these to describe the set of `Counter` objects that will result after a method has completed execution.

Specifying the Counter class

Post-conditions

When we write a unit test for a method we have in our mind some notion of the specification of the method. For example, we could state the specification of the method `Counter.getValue()` in English in the following manner.

> The `getValue()` method returns the value of the `value` instance variable of its `Counter` object. It does not update `value` or `limit`.

The following is a propositional formula with the same meaning.

$$@\text{return} = \text{this.value} \wedge \text{this.value} = \text{this.value}_{prev} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

@return is used to describe the value that the method returns, and the subscript *prev* indicates the value of an instance variable prior to execution of the method. We call this formula the post-condition of the `getValue()` method.

Exercise: Write a post-condition for the `getLimit()` method.

The `increment()` method is different from the other methods because it is a mutator. A post-condition for this method is:

$$\text{this.value} = (\text{this.value}_{prev} + 1) \ \% \ \text{this.limit} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

A post-condition for the constructor is:

$$\text{this.limit} = @\text{limit} \wedge \text{this.value} = @\text{value} \wedge \text{this.value} \geq 0 \wedge \text{this.value} < \text{this.limit} \wedge \text{this.limit} > 0$$

Here we use *@limit* and *@value* to identify the arguments to the method.

Note that the post-condition here is a bit more complicated than the others. Informally, it states that

- The value is greater than or equal to 0 and less than the limit.
- The value and the limit are specified by the two parameters of the constructor.

In general, we try to define our methods so that they have strong post-conditions, i.e. post-conditions which are as informative as possible. For example, the post-condition we have chosen for `getValue()` is better than the simpler post-condition

$$@\text{return} = \text{this.value}$$

Both are correct, but the one we have chosen is more informative because it tells us that the instance variables are not changed. It is important to make sure that the post-condition is not so strong that it is actually incorrect. This seems obvious, but with more complicated methods it can be quite difficult to choose a post-condition that is as strong as possible but not too strong.

Pre-conditions

A post-condition is a *guarantee*, a statement that this is what the method does. The pre-condition of a method is a statement of the method's requirements for correct operation. Together, they form the *contract* that the method has with all its callers. If the caller makes sure that the method's pre-condition is satisfied, then the method guarantees to satisfy the post-condition.

What situation must `getValue()` be in in order to work correctly? Conveniently, `getValue()` will satisfy its post-condition under any circumstances whatsoever. The Boolean value for this situation is $\top$ since this predicate holds for anything.

Exercise: Write the pre-conditions for `getLimit()` and `increment()`.

The pre-condition for the constructor must take into account the fact that it will fail if `limit` is not positive or `value` is negative or `value` is greater than `limit` and is:

$$@\text{limit} > 0 \wedge @\text{value} \geq 0 \wedge @\text{value} < @\text{limit}$$

The whole class

Once the pre-and post-conditions are written for every method in a class it is possible to analyze the class as a whole. Observe that the post-condition for every method asserts that `this.limit` is unchanged. We can deduce from this and the fact that `this.limit` is positive after the constructor has completed execution that `this.limit` is always positive, i.e., that

$$\text{this.limit} > 0$$

is a *class invariant*. Another class invariant is

$$\text{this.value} \geq 0 \wedge \text{this.value} < \text{this.limit}$$

This invariant is obviously maintained by the two "get" methods; it is a little harder to see that it is maintained by the `increment()` method as well.

These two invariants are obviously desirable for a correct `Counter` class; it is good that they appear to hold.

Verifying the pre-conditions

Note that in the previous section we said that the class invariants *appear* to hold. We can not be certain that they hold until we verify that our pre-conditions are correct and that it is impossible to call any method in a state where its pre-condition is not satisfied. Conveniently, the pre-condition for every method in this class is $\top$ so it will always be satisfied. Consequently, it is only necessary to verify that every pre-condition is correct.

Recall that the pre-condition is part of a contract: if the pre-condition is satisfied when the method is called then the post-condition is guaranteed to hold when the method has completed. To make sure that this guarantee will always be satisfied we must examine the body of every method.

Recall that the post-condition for `getValue()` is

$$@\text{return} = \text{this.value} \wedge \text{this.value} = \text{this.value}_{prev} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

The question we must ask is

> What must hold true before the return statement is executed so that, after it is executed, the post-condition holds? Or, in other words, what is the weakest pre-condition of the return statement?

The post-condition is a statement about the value that the method returns. But the value that the method returns is exactly `this.value`. So, we obtain the pre-condition by substituting `this.value` for *@return* getting

$$\text{this.value} = \text{this.value} \wedge \text{this.value} = \text{this.value}_{prev} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

This is the weakest pre-condition for the return statement given the post-condition; since there are no statements before the return statement it is the weakest pre-condition of the method. Note that a weak pre-condition is good; it contains as little information as possible thereby restricting the caller as little as possible. Examining each clause in turn we see that the entire pre-condition is equivalent to $\top$.

$$\text{this.value} = \text{this.value}$$

holds for every `Counter` object.

$$\text{this.value} = \text{this.value}_{prev}$$

holds because there are no preceding statements in the method what might change `this.value`. The same is true for

$$\text{this.limit} = \text{this.limit}_{prev}$$

We need to check that the pre-condition that we have asserted for this method, $\top$, implies, i.e., is no weaker than, the weakest pre-condition that we have just calculated. $\top \rightarrow \top$ so our check succeeds.

Exercise: Verify that the pre-condition for the `getLimit()` method is correct.

Recall that the post-condition for the `increment()` method is:

$$\text{this.value} = (\text{this.value}_{prev} + 1) \ \% \ \text{this.limit} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

To discover the weakest pre-condition that holds before the assignment statement we substitute the value on the right hand side wherever we find `this.value`, getting

$$(\text{this.value} + 1) \ \% \ \text{this.limit} = (\text{this.value}_{prev} + 1) \ \% \ \text{this.limit} \wedge \text{this.limit} = \text{this.limit}_{prev}$$

Since there are no previous statements in the method, $\text{this.value} = \text{this.value}_{prev}$ and $\text{this.limit} = \text{this.limit}_{prev}$. Therefore, the whole expression is equivalent to $\top$ and the pre-condition is verified.

Summary

We have introduced formal specifications as an alternative to testing for describing the behavior of a method and of a class as a whole. Formal specifications are more general than tests, since they make use of logical predicates rather than single values for expressing an expected result. On the other hand, they do not get executed and report errors in the same way as tests. Later we will explore ways to use formal specifications to guide test design and even to automatically generate tests.

Bibliography

[BlueJ] `www.bluej.org`

[Greenfoot] `www.greenfoot.org`

[JUnit] `www.junit.org`

[BK2012] `http://www.bluej.org/objects-first/`

[Checkstyle] `http://checkstyle.sourceforge.net/`